

AI/ML LEARNING ROADMAP

Complete Step-by-Step Guide with Resources

From Foundations to Production Deployment

Advanced ML • Deep Learning • NLP • MLOps • Capstone Projects

Curated Resources • Hands-on Projects • 2026 Edition

PHASE	FOCUS AREA	DURATION
Phase 1	Advanced ML & Ensembles	2-3 Weeks
Phase 2	Deep Learning with PyTorch	4-5 Weeks
Phase 3	NLP & Transformers	3 Weeks
Phase 4	MLOps & Deployment	3 Weeks
Phase 5	Capstone Project	4 Weeks

HOW TO USE THIS GUIDE

This roadmap is designed for learners who have completed foundational Python, mathematics, and basic machine learning concepts. It bridges the gap between academic knowledge and production-ready skills that employers demand.

Key Principles:

- **Code everything yourself** — Close the tutorial and re-implement from memory
- **Consistency over intensity** — 1-2 hours daily beats 10-hour weekend sessions
- **Work with real data early** — Messy real-world data > clean toy datasets
- **Build in public** — Push to GitHub regularly, write about what you learn
- **Track progress** — Use a checklist to mark topics as 'learning', 'understood', or 'practiced'

PREREQUISITES CHECKLIST

Before starting Phase 1, ensure you can confidently:

- Write clean Python scripts and Jupyter notebooks independently
- Set up virtual environments and manage packages with pip
- Perform matrix multiplication, dot products, and vector operations with NumPy
- Load, clean, and visualize datasets with Pandas and Matplotlib
- Explain supervised vs unsupervised learning, overfitting vs underfitting
- Train and evaluate at least two ML models (e.g., Logistic Regression, Decision Tree)
- Understand k-fold cross-validation and basic evaluation metrics

PHASE 1: ADVANCED ML & ENSEMBLE METHODS

Duration: 2-3 Weeks | **Goal:** Master ensemble methods and understand when to use each algorithm

Classical ML still powers a huge portion of production ML systems. Ensemble methods like Random Forest and Gradient Boosting consistently outperform simpler models on structured/tabular data.

STEP 1: XGBoost & LightGBM

XGBoost (eXtreme Gradient Boosting) and LightGBM are industry-standard libraries for gradient boosting. They dominate structured data competitions and production systems.

Resources:

- **Kaggle Learn: Intermediate Machine Learning** — Free, 4 hours. Covers XGBoost, categorical encoding, missing values handling
→ <https://www.kaggle.com/learn/intermediate-machine-learning>
- **XGBoost Official Documentation** — Python API reference and tutorials
→ https://xgboost.readthedocs.io/en/stable/python/python_intro.html
- **LightGBM Documentation** — Microsoft's fast gradient boosting framework
→ <https://lightgbm.readthedocs.io/en/latest/Python-Intro.html>
- **Practice:** Kaggle House Prices Competition — Apply XGBoost with hyperparameter tuning
→ <https://www.kaggle.com/c/house-prices-advanced-regression-techniques>

Key Concepts to Master: Gradient boosting intuition, learning rate, max_depth, n_estimators, early stopping, handling categorical features

STEP 2: Random Forest Deep Dive

Random Forest uses bagging (Bootstrap Aggregating) to create decorrelated trees. Understanding this helps you debug and tune ensemble models effectively.

Resources:

- **Scikit-learn: Ensemble Methods** — Comprehensive guide with code examples
→ <https://scikit-learn.org/stable/modules/ensemble.html>
- **StatQuest YouTube:** 'Random Forests' and 'Gradient Boosting' playlists — Visual, intuitive explanations
→ <https://www.youtube.com/c/joshstarmer>
- **Practice:** Compare Random Forest vs XGBoost on the same dataset. Document differences in accuracy, training time, and feature importance

STEP 3: SVMs, PCA & Unsupervised Learning

Support Vector Machines provide maximum-margin classification. PCA reduces dimensionality while preserving variance. K-Means and DBSCAN discover hidden patterns in unlabeled data.

Resources:

- **CS229 Andrew Ng (Stanford)** — SVM lectures (theoretical foundation, free)
→ <https://cs229.stanford.edu/syllabus.html>
- **Scikit-learn: Decomposition** — PCA implementation and examples
→ <https://scikit-learn.org/stable/modules/decomposition.html>
- **Scikit-learn: Clustering** — K-Means, DBSCAN, and evaluation metrics
→ <https://scikit-learn.org/stable/modules/clustering.html>
- **Practice:** Implement PCA from scratch with NumPy, then compare results with `sklearn.decomposition.PCA`

Deliverable: Jupyter notebook comparing 4 algorithms (Random Forest, XGBoost, SVM, Logistic Regression) on a dataset with full evaluation metrics

PHASE 2: DEEP LEARNING WITH PYTORCH

Duration: 4-5 Weeks | **Goal:** Build and train neural networks from scratch, understand CNNs for computer vision

Deep learning is the backbone of modern AI. Start by understanding fundamentals — how neurons work, how networks learn, and how to implement this in PyTorch. Avoid the black-box mindset.

STEP 1: PyTorch Fundamentals

PyTorch is the standard framework for deep learning research and production. Master tensors, autograd, and the nn.Module system before building complex models.

Resources:

- **PyTorch 60-Minute Blitz** — Official quickstart, essential first step (Free)
→ https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html
- **Fast.ai: Practical Deep Learning for Coders** — Top-tier practical course, Part 1 (Free)
→ <https://course.fast.ai/>
- **Book: 'Deep Learning with PyTorch'** by Eli Stevens, Luca Antiga — Chapters 1-8
→ Available on Amazon / Manning Publications
- **PyTorch Official Tutorials** — Comprehensive beginner to advanced examples
→ <https://pytorch.org/tutorials/>
- **Practice:** Build a linear regression model in PyTorch from scratch using torch.nn and torch.optim

STEP 2: Neural Networks from Scratch

Understanding backpropagation and training loops at a low level is non-negotiable. Build a 2-layer network with NumPy first, then rebuild in PyTorch.

Resources:

- **Andrej Karpathy: 'Neural Networks: Zero to Hero'** — Best video series on fundamentals (Free)
→ <https://www.youtube.com/playlist?list=PLAqhlrjxibuWI23v9cThsA9GvCAUhRvKZ>
- **3Blue1Brown: Neural Networks Series** — Visual intuition for backpropagation
→ https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- **Practice Project:** Implement a 2-layer neural network using ONLY NumPy + math
→ Then rebuild using PyTorch nn.Module, autograd, and DataLoader
- **Key Concepts:** Perceptron, activation functions (ReLU, sigmoid, tanh), forward pass, chain rule, gradient descent, SGD, Adam

STEP 3: CNNs & Computer Vision

Convolutional Neural Networks (CNNs) are the standard architecture for image tasks. Understanding CNNs also helps grasp the intuition behind attention and transformers.

Resources:

- **Stanford CS231n: Convolutional Neural Networks** — The definitive CNN course (Free)

→ <http://cs231n.stanford.edu/>

- **PyTorch Transfer Learning Tutorial** — Fine-tuning pretrained models

→ https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

- **PyTorch CNN Tutorial** — CIFAR-10 classifier with data augmentation

→ https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

- **Practice Projects:**

1. MNIST classifier — Target: >95% accuracy
2. CIFAR-10 classifier — Target: >75% accuracy (use data augmentation)
3. Transfer learning project — Use ResNet or EfficientNet on custom dataset

Key Concepts: Convolution operation, filters, feature maps, stride, pooling (max/avg), batch normalization, dropout, data augmentation, transfer learning vs fine-tuning

PHASE 3: NLP & TRANSFORMERS

Duration: 3 Weeks | **Goal:** Master text processing, classical NLP, and modern Transformer architectures

NLP is one of the most active areas of AI research. Start with fundamentals — text preprocessing and classical NLP — before moving to modern Transformer-based models.

STEP 1: Classical NLP & Text Processing

Before Transformers, text was represented using Bag-of-Words, TF-IDF, and word embeddings. These foundations are still relevant for understanding modern approaches.

Resources:

- **Kaggle Learn: Natural Language Processing** — Free, 4-hour course covering TF-IDF, word vectors
→ <https://www.kaggle.com/learn/natural-language-processing>
- **NLTK Book** — Free online textbook for classical NLP
→ <https://www.nltk.org/book/>
- **Spacy Documentation** — Industrial-strength NLP library
→ <https://spacy.io/usage>
- **Practice:** Build a sentiment classifier using TF-IDF + Logistic Regression on IMDB dataset

STEP 2: Word Embeddings

Word embeddings (Word2Vec, GloVe) represent words as dense vectors capturing semantic meaning. This bridges classical NLP and deep learning approaches.

Resources:

- **TensorFlow Word2Vec Tutorial** — Implement Word2Vec from scratch
→ <https://www.tensorflow.org/tutorials/text/word2vec>
- **GloVe Pretrained Vectors** — Stanford NLP Group
→ <https://nlp.stanford.edu/projects/glove/>
- **Practice:** Visualize word embeddings with t-SNE. Find analogies (king - man + woman = queen)

STEP 3: Transformers & HuggingFace

The Transformer architecture (Vaswani et al., 2017) revolutionized NLP. Self-attention mechanisms allow models to process entire sequences in parallel. HuggingFace provides easy access to pretrained models.

Resources:

- **Paper: 'Attention Is All You Need'** — Read the architecture section carefully
→ <https://arxiv.org/abs/1706.03762>
- **HuggingFace NLP Course** — Comprehensive free course on Transformers
→ <https://huggingface.co/learn/nlp-course/chapter1/1>
- **HuggingFace Transformers Documentation** — API reference and examples
→ <https://huggingface.co/docs/transformers/index>
- **Jay Alammar: 'The Illustrated Transformer'** — Visual guide to attention mechanism
→ <https://jalammar.github.io/illustrated-transformer/>
- **Practice:** Fine-tune BERT for sentiment analysis using HuggingFace Trainer API
→ Dataset: [IMDB Reviews](#) or [Twitter Sentiment Analysis](#)

Key Concepts: Tokenization, attention mechanism, self-attention, multi-head attention, positional encoding, BERT, fine-tuning vs feature extraction

PHASE 4: MLOPS & DEPLOYMENT

Duration: 3 Weeks | **Goal:** Deploy models to production with proper tooling and reproducibility

Building good models is only half the job. You also need to track experiments, version code, and serve models. These skills separate academic ML from production-ready engineering.

STEP 1: Experiment Tracking

Track hyperparameters, metrics, and artifacts systematically. Weights & Biases (W&B;) offers superior UX compared to MLflow for individual learners.

Resources:

- **Weights & Biases (W&B;) Quickstart** — Free tier, excellent for beginners
→ <https://docs.wandb.ai/quickstart>
- **W&B; PyTorch Integration** — Log training loops automatically
→ <https://docs.wandb.ai/guides/integrations/pytorch>
- **MLflow Documentation** — Open-source alternative (Databricks)
→ <https://mlflow.org/docs/latest/index.html>
- **Practice:** Add W&B; logging to your PyTorch CNN training script. Log loss curves, accuracy, and hyperparameters

STEP 2: REST API Deployment

Serve trained models via web endpoints. FastAPI is the modern standard — async, automatic documentation, and type hints. Flask is simpler for small projects.

Resources:

- **FastAPI Official Tutorial** — Complete from basics to advanced (3 hours)
→ <https://fastapi.tiangolo.com/tutorial/>
- **Deploy ML Models with FastAPI** — YouTube tutorial by Patrick Loeber
→ <https://www.youtube.com/watch?v=1zMQBe0l1bM>
- **Flask Quickstart** — If you prefer a simpler framework
→ <https://flask.palletsprojects.com/en/2.3.x/quickstart/>
- **Practice:** Build a FastAPI app that accepts JSON input and returns model predictions
→ Test with curl: `curl -X POST 'http://localhost:8000/predict' -H 'Content-Type: application/json' -d '{"features": [1,2,3]}'`

STEP 3: Docker & Containerization

Containerize your ML application for consistent deployment across environments. Docker is the industry standard for packaging dependencies.

Resources:

- **Docker for Beginners** — Interactive tutorial (Free)
→ <https://docker-curriculum.com/>
- **Docker Official Documentation** — Best practices for Python apps
→ <https://docs.docker.com/language/python/>
- **Practice:** Create a Dockerfile for your FastAPI app. Build and run: `docker build -t ml-api . && docker run -p 8000:8000 ml-api`

STEP 4: Model Serialization & Advanced Deployment

Save and load models efficiently. Consider ONNX for cross-platform deployment and TorchScript for PyTorch production.

Resources:

- **PyTorch: Save/Load Models** — Best practices for model persistence
→ https://pytorch.org/tutorials/beginner/saving_loading_models.html
- **ONNX Runtime** — Cross-platform accelerated inference
→ <https://onnxruntime.ai/>
- **Scikit-learn: Model Persistence** — joblib for pipelines
→ https://scikit-learn.org/stable/model_persistence.html
- **Practice:** Export your trained PyTorch model to ONNX and benchmark inference speed vs native PyTorch

Deliverable: A complete ML API in a Docker container with W&B; experiment tracking and model serialization

PHASE 5: CAPSTONE PROJECT

Duration: 4 Weeks | **Goal:** Build and deploy a complete end-to-end ML system

The capstone is your most important deliverable. It demonstrates genuine, employable skills to future employers, universities, and collaborators. This is what separates learners from practitioners.

CAPSTONE PROJECT IDEAS

PROJECT	SKILLS DEMONSTRATED	DATASET SOURCE
House Price Predictor + API	Regression, XGBoost, feature engineering, FastAPI deployment	Kaggle: House Prices
Medical Image Classifier	CNNs, transfer learning, data augmentation, Docker	Kaggle: Chest X-Ray or Pneumonia
Sentiment Analysis API	NLP, BERT fine-tuning, HuggingFace, REST API	IMDB Reviews / Twitter API
Customer Churn Predictor	Classification, ensembles, MLOps, W&B tracking	Kaggle: Telco Customer Churn
Stock Price Predictor	Time series, LSTM/GRU, deployment, monitoring	Yahoo Finance API / Kaggle

CAPSTONE CHECKLIST

Your repository must include:

■ Project Structure

- /data — Raw and processed datasets
- /models — Saved model artifacts
- /api — FastAPI application code
- /notebooks — EDA and experimentation Jupyter notebooks
- /src — Modular Python scripts for training and preprocessing

■ Code Quality

- Clean, well-commented code with docstrings
- requirements.txt with pinned dependency versions
- README.md with architecture diagram and setup instructions

■ Data Pipeline

- Data collection (scraping, API, or curated dataset)
- Full EDA with visualizations (correlation heatmaps, distributions)
- Feature engineering with clear rationale

■ Modeling

At least 3 different models trained and compared
Hyperparameter tuning (GridSearchCV, Optuna, or manual)
Evaluation on held-out test set with full metrics
Experiment tracking with W&B; or MLflow

■ **Deployment**

REST API with FastAPI serving predictions
Docker containerization
Model serialization (pickle, ONNX, or TorchScript)

■ **Documentation**

Clear README with project overview and results
Jupyter notebook walkthrough for non-technical audiences
Short project report (2-3 pages) explaining methodology

WEEKLY STUDY SCHEDULE

Recommended for part-time learners (working/studying alongside): 1-2 hours daily with focused weekend sessions. Adjust based on your availability.

DAY	FOCUS	DURATION	ACTIVITY
Monday	Theory / Video Lectures	1-2 hrs	Watch courses, read papers, take notes
Tuesday	Code Implementation	1-2 hrs	Implement concepts from scratch in Jupyter
Wednesday	Theory / Reading	1-2 hrs	Read documentation, blog posts, book chapters
Thursday	Code / Project Work	1-2 hrs	Build mini-projects, debug, experiment
Friday	Kaggle / Practice	1-2 hrs	Compete on Kaggle, solve practice problems
Saturday	Project Build Day	3-4 hrs	Deep work on capstone or significant projects
Sunday	Review / Rest	Flexible	Review week's learning, plan next week, or rest

CRITICAL SUCCESS METRICS

By the end of this roadmap, you should be able to:

1. Implement a neural network in PyTorch without looking at documentation

Write forward pass, loss computation, backpropagation, and optimizer step from memory.

2. Explain the difference between bagging and boosting with examples

Know when to use Random Forest vs XGBoost vs LightGBM on tabular data.

3. Deploy a model via FastAPI and successfully curl it

Build a containerized API that accepts JSON and returns predictions with <100ms latency.

4. Fine-tune BERT for a custom text classification task

Use HuggingFace Transformers to load pretrained weights, add classification head, and train.

5. Submit a top-50% score on a Kaggle competition

Understand competition workflow, leaderboard evaluation, and iterative improvement.

6. Debug a failing model

Identify overfitting/underfitting, learning rate issues, and data leakage.

7. Read and understand an introductory ML research paper

Follow the math and architecture diagrams in papers like Attention Is All You Need.

IMMEDIATE NEXT STEPS

This Week:

1. Start **PyTorch 60-Minute Blitz** — Complete tensors and autograd sections
2. Create a new GitHub repository called 'ml-learning-journey'
3. Set up a Kaggle account and join the House Prices competition

Next Week:

1. Complete your first XGBoost model on House Prices dataset
2. Submit to Kaggle leaderboard (any score is fine for first attempt)
3. Write a blog post or README explaining what you learned

Month 1 Goal:

- One PyTorch model trained and saved
- One Kaggle submission on GitHub
- One FastAPI endpoint running locally
- W&B; account with at least 3 logged experiments

Month 3 Goal:

- Complete CNN project (MNIST or CIFAR-10) with >90% accuracy
- Fine-tune BERT for text classification
- Docker container running your API

Month 6 Goal:

- Complete capstone project with full deployment
- Portfolio of 3+ projects on GitHub
- Active Kaggle profile with ranked submissions

ESSENTIAL TOOLS & PLATFORMS

CATEGORY	TOOL	PURPOSE	LINK
Code	VS Code + Jupyter	Development environment	code.visualstudio.com
Version Control	Git + GitHub	Code versioning & portfolio	github.com
Cloud GPU	Google Colab	Free GPU for deep learning	colab.research.google.com
Cloud GPU	Kaggle Notebooks	Free GPU + competitions	kaggle.com
Experiment Tracking	Weights & Biases	Log metrics & hyperparameters	wandb.ai
Experiment Tracking	MLflow	Open-source alternative	mlflow.org
Deployment	FastAPI	Modern Python web framework	fastapi.tiangolo.com

Deployment	Docker	Containerization	docker.com
Data	Kaggle Datasets	Real-world datasets	kaggle.com/datasets
Data	HuggingFace Datasets	NLP datasets & models	huggingface.co/datasets
Data	UCI ML Repository	Classic ML datasets	archive.ics.uci.edu/ml
Community	r/MachineLearning	Reddit community	reddit.com/r/MachineLearning
Community	Papers With Code	Papers + implementations	paperswithcode.com

STUDY TIPS FOR SUCCESS

1. Consistency Over Intensity

Studying for 1-2 hours every day is far more effective than 10-hour weekend sessions. Build a daily habit — even 45 minutes of focused practice compounds quickly over 12 months.

2. Code Everything Yourself

Reading or watching tutorials is not the same as learning. After every concept, close the tutorial and re-implement it from memory. Only hands-on practice builds real intuition.

3. Do Not Skip the Mathematics

Many students skip the math and learn ML as a collection of API calls. This works until it doesn't — when debugging fails or when you need to understand why a model is not converging. Invest time in linear algebra and calculus foundations.

4. Work With Real Data Early

Kaggle, UCI ML Repository, and HuggingFace Datasets all have freely available real-world datasets. Using messy, real data instead of clean toy datasets prepares you for actual industry work.

5. Build in Public

Push your code and notebooks to GitHub regularly. Write short blog posts or Notion pages explaining what you learned. Teaching others forces you to understand concepts at a deeper level.

6. Track Your Progress

Use a simple checklist or Notion board to mark topics as 'learning', 'understood', or 'practiced'. Revisit topics marked 'understood' at least once every two weeks to prevent forgetting.

7. Embrace the Struggle

Debugging a failing model for 3 hours teaches you more than watching 10 tutorials. The frustration is part of the learning process. Document your mistakes — they become your

best teachers.

Remember: The goal is not to know everything — it is to build a solid, practical foundation and develop the habit of learning continuously. Every expert was once a beginner. Follow the plan, stay consistent, and the results will follow.

— Good luck on your AI/ML journey —